

Project 3: Production Application with Claude Code Mastery

[Start Assignment](#)

- Due Apr 22 by 2:59am
- Points 200
- Submitting a text entry box, a website url, or a file upload

Project 3: Production Application with Claude Code Mastery

Weight: 19% of final grade | **Points:** 200

Objective

Build a production-grade, deployed application as a pair, demonstrating mastery of Claude Code's extensibility features, professional AI-assisted workflows, and production engineering practices taught in W10-W14.

Approval Requirement

Project idea must be approved by the professor on the #projects Slack channel at least one week before the deadline.

Requirements

Functional Requirements

- Production-ready application solving a real problem
- 2+ user roles or distinct feature areas
- Real-world use case (new idea)
- Portfolio/interview-worthy quality
- Deployed and accessible via public URL

Technical Requirements

Architecture

- Next.js full-stack application (App Router or Pages Router)
- Database (PostgreSQL recommended, or equivalent)
- Authentication (Auth.js/NextAuth, Clerk, or equivalent)
- Deployed on Vercel (or equivalent platform with preview deploys)

Claude Code Mastery (core of this project)

Each of the following Claude Code concepts must be demonstrated with evidence in your repository:

CLAUDE.md & Memory (W10):

- Comprehensive CLAUDE.md with @imports for modular organization
- Auto-memory usage for persistent project context
- Evidence of CLAUDE.md evolution across the project (visible in git history)
- Project conventions, architecture decisions, and testing strategy documented

Custom Skills (W12) — minimum 2:

- At least 2 skills in `.claude/skills/` (e.g., `/fix-issue`, `/add-feature`, `/deploy`, `/create-pr`)
- Evidence of team usage (session logs or screenshots)
- At least one skill iterated from v1 to v2 based on real usage

Hooks (W12) — minimum 2:

- At least 2 hooks configured in `.claude/settings.json`
- At least one PreToolUse or PostToolUse hook (e.g., auto-format, block protected files, lint-on-edit)
- At least one quality-enforcement hook (e.g., Stop hook that runs tests)

MCP Servers (W12) — minimum 1:

- At least 1 MCP server integrated (database, Playwright, GitHub, or other)
- Configuration shared via `.mcp.json` in repository
- Evidence of use in development workflow (session logs or screenshots)

Agents (W12-W13) — minimum 1 (choose any):

- Custom sub-agents in `.claude/agents/` (e.g., security-reviewer, test-writer), OR
- Agent teams with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`, OR
- Agent SDK feature built into the application (applying W13 patterns)
- Evidence of use (session log, PR, or screenshots showing agent output)

Parallel Development (W12):

- Evidence of worktree usage for parallel feature development
- At least 2 features developed in parallel (visible in git branch history)

Writer/Reviewer Pattern + C.L.E.A.R. (W12):

- At least 2 PRs using the writer/reviewer pattern (one agent writes, another reviews)
- C.L.E.A.R. framework applied in PR reviews (visible in PR comments)
- AI disclosure metadata in PRs (% AI-generated, tool used, human review applied)

Test-Driven Development (W11)

- TDD workflow (red-green-refactor) for at least 3 features
- Git history showing failing tests committed before implementation
- Unit + integration tests (Vitest or Jest)
- At least 1 E2E test (Playwright)
- 70%+ test coverage

CI/CD Pipeline (W14) — GitHub Actions

- Lint (ESLint + Prettier)
- Type checking (tsc --noEmit)
- Unit and integration tests
- E2E tests (Playwright)
- Security scan (npm audit)
- AI PR review (claude-code-action or claude -p)
- Preview deploy (Vercel)
- Production deploy on merge to main

Security (W14) — minimum 4 gates from the 8-gate pipeline

- Pre-commit secrets detection (Gitleaks or equivalent)
- Dependency scanning (npm audit in CI)
- At least one SAST tool or security-focused sub-agent
- Security acceptance criteria in Definition of Done
- OWASP top 10 awareness documented in CLAUDE.md

Team Process

- 2 sprints documented (sprint planning + retrospective each)
- GitHub Issues with acceptance criteria as testable specifications
- Branch-per-issue workflow with PR reviews
- Async standups (minimum 3 per sprint per partner)
- C.L.E.A.R. framework applied in PR reviews
- Peer evaluations

Deliverables

1. GitHub repository with full `.claude/` configuration (skills, hooks, agents, MCP)
2. Deployed application (Vercel production URL)
3. CI/CD pipeline (GitHub Actions, all stages passing)
4. Technical blog post (published on Medium, dev.to, or similar)
5. Video demonstration (5-10 min, showcasing app + Claude Code workflow)
6. Individual reflections (one per partner, 500 words)
7. **Showcase submission** via [Google Form](https://docs.google.com/forms/d/e/1FAIpQLScT67tnwjhlETSRwADt57TS_THJSeSGf-xrjTV2nm-XvfFELg/viewform?usp=dialog)  (https://docs.google.com/forms/d/e/1FAIpQLScT67tnwjhlETSRwADt57TS_THJSeSGf-xrjTV2nm-XvfFELg/viewform?usp=dialog) (project name, URLs, thumbnail, video, blog)

Rubric (200 points)

Category	Points	Description
Application Quality	40	Production-ready, deployed, polished, real use case
Claude Code Mastery	55	Skills, hooks, MCP, agents, CLAUDE.md/memory, worktrees, C.L.E.A.R.
Testing & TDD	30	TDD workflow, coverage, test pyramid

CI/CD & Production	35	Pipeline stages, AI review, Vercel deploy, security gates
Team Process	25	Sprints, PRs, C.L.E.A.R. reviews, async standups, peer evals
Documentation & Demo	15	README, blog post, video demo, reflections

Bonus (up to 10 extra points): Property-based testing with fast-check (+3), Mutation testing with Stryker (+3), Agent SDK feature applying W13 patterns (+4)

Note: Individual grades adjusted by peer evaluations ($\pm 10\%$)

Project 3: Production Application with Claude Code Mastery Rubric

Criteria	Ratings					Pts
Application Quality	40 pts Excellent - Production-ready, deployed on Vercel, polished UI, 2+ user roles, real problem solved, portfolio- worthy	30 pts Good - Deployed, core features work, good UX, minor gaps	20 pts Satisfactory - Functional app, basic UI, partially deployed	10 pts Needs Improvement - Incomplete features, not deployed or broken	0 pts Unsatisfactory - Major functionality broken or missing	40 pts
Claude Code Mastery	55 pts Excellent - Rich CLAUDE.md with @imports and git evolution; 2+ iterated skills with usage evidence; 2+ hooks enforcing quality; MCP server integrated via .mcp.json; agents (sub- agents/teams/SDK) with evidence; parallel worktree development; 2+ PRs with writer/reviewer + C.L.E.A.R. + AI disclosure	42 pts Good - Functional CLAUDE.md with iteration; 2 skills and 2 hooks configured; MCP and agents present with some usage evidence; some parallel development; C.L.E.A.R. applied on some PRs	28 pts Satisfactory - Basic CLAUDE.md; 1-2 skills/hooks with limited use; MCP or agents configured but minimal evidence; limited parallel work or C.L.E.A.R. usage	14 pts Needs Improvement - Minimal CLAUDE.md; missing multiple Claude Code features (skills, hooks, MCP, or agents); no parallel development or C.L.E.A.R. evidence	0 pts Unsatisfactory - No meaningful Claude Code extensibility demonstrated	55 pts
Testing & TDD	30 pts	22 pts	15 pts	8 pts	0 pts	30 pts

Criteria	Ratings					Pts
	Excellent - TDD red-green-refactor for 3+ features visible in git; 70%+ coverage; unit + integration + E2E (Playwright); tests verify behavior and edge cases	Good - TDD for some features; adequate coverage; unit + integration tests present; functional test quality	Satisfactory - Some tests written (may be after code); low coverage; missing test types	Needs Improvement - Minimal tests, no TDD evidence, trivial assertions	Unsatisfactory - No meaningful testing	
CI/CD & Production	35 pts Excellent - All 8 pipeline stages green (lint, typecheck, tests, E2E, security, AI review, preview, prod deploy); 4+ security gates; Sentry with source maps + structured logging; issue diagnosed via monitoring	26 pts Good - 5-7 pipeline stages; AI review configured; 3 security gates; Sentry configured with some logging	18 pts Satisfactory - 3-4 pipeline stages; partial security scanning; basic error tracking	9 pts Needs Improvement - Fewer than 3 stages; minimal security; no monitoring	0 pts Unsatisfactory - No CI/CD pipeline or production infrastructure	35 pts
Team Process	25 pts Excellent - 2 sprints with planning + retrospectives; branch-per-issue with PR reviews; 3+ async standups/sprint/partner; C.L.E.A.R. in reviews; AI disclosure; thoughtful peer evaluation	19 pts Good - 2 sprints documented; branch workflow with some PR reviews; standups present; peer evaluation completed	13 pts Satisfactory - Basic sprint docs; some branching; minimal standups or reviews	6 pts Needs Improvement - Incomplete sprint documentation; no structured workflow	0 pts Unsatisfactory - No team process documentation	25 pts

Criteria	Ratings					Pts
Documentation & Demo	15 pts Excellent - Clear README with Mermaid architecture diagram; published blog post with AI workflow insights; polished 5-10 min screencast; 500-word reflections with specific Claude Code insights; smooth 8-10 min live demo following W14 structure	11 pts Good - README present; blog post published; screencast covers main points; adequate reflections; demo mostly follows structure	8 pts Satisfactory - Minimal README; draft blog or shallow screencast; short reflections; demo over/under time	4 pts Needs Improvement - Missing multiple deliverables; weak demo	0 pts Unsatisfactory - Missing major documentation or no demo	15 pts
Total Points: 200						